

資料結構 (CO2012) 作業實作報告

Stack、Function Pointer 與 Event-Driven Calculator (v1.1.0)

10/15/2026

林郁翔 (114503530)

Department of Communication Engieering, NCU

Contents

1. AI 使用聲明 **1**

2. 作業要求摘要 **1**

3. 實作成果說明 **1**

3.1. C 語言中的 Non-blocking 程式設計方式 2

3.2. 同時維持使用者輸入、時間顯示板更新之設計模式 3

3.3. Operand 與 Operator Stack 設計 3

3.4. 各二元實數運算函式設計 4

3.5. Function Pointer 之使用 4

3.6. `libev` 之 Event Loop 使用 5

3.7. 回呼函式 (Callback Function) 功能說明 6

3.7.1. `static void calc_cb (EV_P_ ev_io *w, int revents)` 6

3.7.2. `static void terminated_cb(EV_P_ ev_signal *w, int revents)` 6

3.7.3. `static void resize_cb(EV_P_ ev_signal *w, int revents)` 6

3.7.4. `static void timer_cb(EV_P_ ev_timer *w, int revents)` 6

4. 複雜度分析 **7**

4.1. 空間複雜度分析 7

4.2. 時間複雜度 7

5. 測試結果 **7**

5.1. 基本測試資料與結果 7

5.2. 待修復問題 8

1. AI 使用聲明

本作業及其衍生報告及內容，部分使用 OpenAI 公司之產品（包含 ChatGPT、Codex）進行輔助開發與製作（具體用途如下列）。本作業作者對所有 AI 所輔助完成之內容產出負一切責任。

- 原始碼部分運作邏輯發想與修正。
- 一般狀況及極端狀況程式測試。
- 文件統整、資料閱讀。

本作業實作時，不直接以 AI 工具生成任何原始碼，僅針對 debugging、理論，即所有原始碼內容均由本人閱讀並確認 AI 生成相關說明及範例程式碼後，自行編修並寫出。

2. 作業要求摘要

本作業要求以 C 語言實作使用 libev 的非阻塞事件驅動計算器，並有以下要求：

- 使用自行實作的運算元堆疊與運算子堆疊計算中序 (infix) 算式。
- 支援四則運算、多位數整數、空白忽略，以及正確的運算優先順序與左結合。
- 透過函式指標呼叫各自獨立的四則運算函式。
- 使用 `libev` 同時處理時間更新與標準輸入：每秒更新固定位置的時間，並在使用者輸入算式後計算結果。
- 支援連續輸入、錯誤處理及結束指令。
- 繳交原始碼、Makefile、至少五組測試結果，以及程式設計與複雜度分析說明。

3. 實作成果說明

現正持續進行開發之 GitHub Repository： https://github.com/Spig100/comm_ds-calc_with_libev

本專案之開發環境為 Debian 13.7 (trixie) Linux 發行版，配合 Vim 編輯器與其 syntax 插件、 GCC 編譯工具以及 `libev-dev` 包進行開發與編譯。

值得注意的是，為了確保列印結果可控，且不破壞原有 shell 的紀錄導致畫面雜亂，此專案中額外使用了備用螢幕 (alternate screen) 作為專案運行位置，以利後續實作時間顯示板、固定行數滾動免於破壞時間板等功能。

3.1. C 語言中的 Non-blocking 程式設計方式

由於本作業中要求以 `libev` 製作同時具備時間顯示板、計算機功能的 C 程式，故此處特別說明以 `libev` 此一事件迴圈導向的套件庫，為何能達成 non-blocking 之效果。

常人所能想到最符合實務的處理方法，常是將各個進程獨立跑在不同的線程 (thread) 上。但是由於 C 語言的特性，其並不具備如 Java、Python、JavaScript 等現代調用多執行緒、進行異步 (async) 程式的能力。以本次作業之需求為例，若通過一 `while` 迴圈持續監聽使用者的算式輸入，將因迴圈阻塞目前線程，無法在等待同時持續更新時間顯示板。

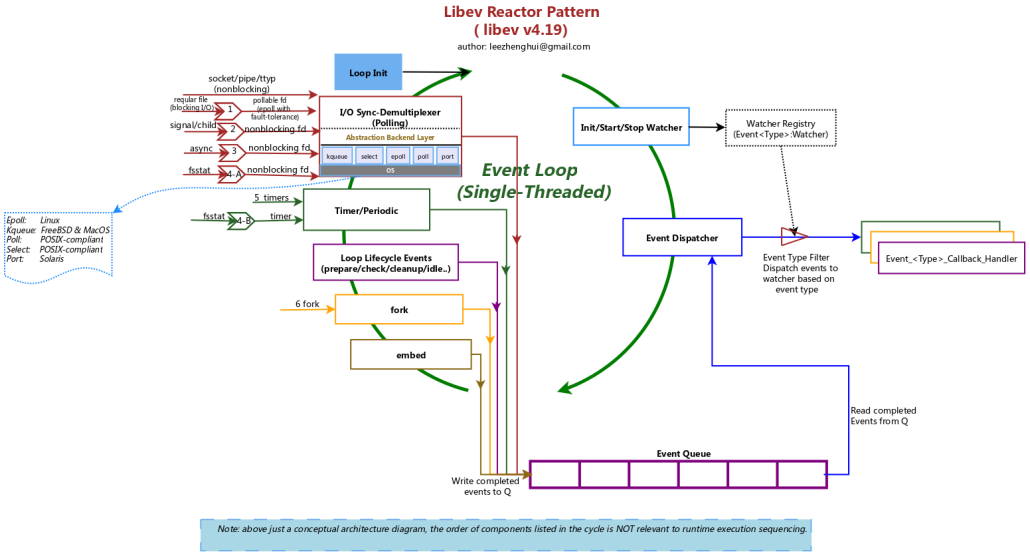


圖 1　`libev` 的運作圖像[1]。

此時引入 `libev` 這個以事件迴圈 (event loop) 設計的工具便相當合適——作為前述 `while` 迴圈的替代品。一般的 `libev` 主程式運作流程是：

1. 指定待觀測的事件（如輸入輸出、計時器），設定對應的事件觀測器 (watcher)；watcher 本身也會配置一個對應的回呼函式 (callback function)。
2. 將觀測器註冊到 event loop 中。
3. 啟動事件迴圈。

每一次的 event loop 循環中，將逐步通過各 watcher 設定條件判斷事件是否被觸發，若有則呼叫對應的回呼函式進行操作並等待；操作完成則繼續循環。

如此的運作流程，雖本質仍為單線程執行，但各個功能（輸入輸出處理、計時器監測）並未直接佔據線程使進程阻塞，即可達成各個功能互相的 non-blocking。

[1] https://leezhenghui.github.io/io-strategy/2018/12/01/io-strategy-dive-into-libev-libeio.html

值得注意的是， event loop 及其中的 callback functions，仍在單一線程執行。若單一 callback function 致使線程阻塞，仍可能導致整體進程遭到拖延。總而言之，雖其可通過上段所述的工作流達成各事件監測的 non-blocking，但若 callback function 設計不當，仍可能使 event loop 遭到阻塞，進而使得其他進程受到影響。

3.2. 同時維持使用者輸入、時間顯示板更新之設計模式

如小節 3.1 所述，若可以通過註冊對應事件的 watcher，即可利用 `libev` event loop 的特性，達成兩功能互不阻塞的效果。具體在程式中的實作請見小節 3.6 的詳細說明。

3.3. Operand 與 Operator Stack 設計

本專案中自行實作了使用雙重浮點數 (`double`) 與字元 (`char`) 為內容之堆疊 (stack)：

- `typedef struct num_stack`
- `typedef struct char_stack`

堆疊內容以固定長度 128 之 C 語言預設陣列 (array) 實作，並以一 8 bit 有號整數 `int8_t`（於 $[-128, 127]$ 之所有整數）之值 `top` 作為索引。具體實作之程式碼如下（記憶體使用狀況如後）。：

```c
/* "src/stack.h" Ln 17-20, 32-35 */

typedef struct {
    int8_t top;                         // 1 B
    double content[STACK_MAX_SIZE];     // 8 B
} num_stack;                            // 9 B in total

typedef struct {
    int8_t top;                         // 1 B
    char content[STACK_MAX_SIZE];       // 1 B
} char_stack;                           // 2 B in total
```

另外，針對此資料結構，實作了以下函式。此處 `type_stack` 代表前述特定型別 `type` 的 stack，因資料類型不同但實質功能相同，合併列舉之。

- 元素初始化： `void type_stack_init (type_stack * const stackPtr);`
- 檢查是否為空： `bool type_stack_isEmpty (const type_stack * const stackPtr);`
- 檢查是否溢出： `bool num_stack_isOverflow (const num_stack * const stackPtr);`
- 彈出 (pop)： `type type_stack_pop (num_stack * const stackPtr);`
- 推入 (push)： `uint8_t type_stack_push (num_stack * const stackPtr, type x);`

- 得到最上層元素但不彈出 (top)： `type type_stack_top (const num_stack * const stackPtr);`

以上功能之實作均以指針 (pointer) 方式進行位址傳遞 (call by address)，以直接修改在主程式中所宣告的對應 stack。

3.4. 各二元實數運算函式設計

參考本作業之要求：

> 當程式需要執行某個運算時，必須先根據 Operator 取得對應的 Function Pointer，再透過該 Function Pointer 呼叫相對應的運算函式。

此專案將運算函式統一型別 `Operation`，為一輸入兩 `double`，輸入一 `double` 的函式指標 (function pointer)。

```
typedef double (*Operation) (double, double);
```

另外共實作了以下分開的計算函式。注意到本專案將所有未定義之函數以 “Undefined” 進行處理，故針對除零狀況回傳 `NaN`。

```
1  /* "src/calc.c" Ln 11-30 */
2
3  double add (double x, double y){
4      return x + y;
5  }
6
7  double substract (double x, double y){
8      return x - y;
9  }
10
11 double multiply (double x, double y){
12     return x * y;
13 }
14
15 double divide (double x, double y){
16     if (y == 0) return NAN;
17     return x / y;
18 }
19
20 double power (double x, double y){
21     return pow(x,y);
22 }
```

3.5. Function Pointer 之使用

實作一型別為 `Operation` 之函數 `get_operation(char)`，通過 switch-case 語法結構依據輸入之運算元 (operator) 進行判斷，回傳對應的運算函數，並進而操作之。

```c
/* "src/calc.c" Ln 32-53 */

Operation get_operation (char opr){
    switch(opr){
        case '+':
            return add;
            break;
        case '-':
            return substract;
            break;
        case '*':
            return multiply;
            break;
        case '/':
            return divide;
            break;
        case '^':
            return power;
            break;
        default:
            return NULL;
            break;
    }
}
```

使用狀況如 `int flush_operator(num_stack * const operand, char_stack * const operator)`，進行獲取到的對應運算 function pointer 後直接傳值進行操作。

```c
/* "src/calc.c" Ln 62-73 */

int flush_operator(num_stack * const operand, char_stack * const operator){
    char op = char_stack_pop(operator);
    //fprintf(stderr, "Debug: op flush start - %c\n", op);
    Operation operation = get_operation(op);
    if (operation == NULL) return -1;
    double b = num_stack_pop(operand);
    double a = num_stack_pop(operand);
    if (a == NAN || b == NAN) return -1;
    num_stack_push(operand, operation(a, b));
    //fprintf(stderr, "DEBUG: %g %c %g\n", a, op, b);
    return 0;
}
```

3.6. libev 之 Event Loop 使用

本專案中利用 libev 函式庫進行事件 (event) 觸發導向流程設計。其中，由於本次需實作功能僅涉及單一事件迴圈 (event loop) 之操作，故使用以下方式宣告，使用 libev 所做巨集中預設的事件迴圈：

```c
struct ev_loop *loop = EV_DEFAULT;
```

本專案主要使用以下類型的 event 監聽器，並通過函式回呼 (function callback) 方式定義事件發生後的下一步操作：

- 輸入輸出流監聽器 `ev_io`： 針對標準輸入流 `stdin` 訊號進行偵測，觸發時 callback `calc_cb` 函式進行運算。
- 時鐘監聽器 `ev_timer`：對系統時鐘更新進行偵測，觸發時 callback `timer_cb`
- 系統訊號接收器 `ev_signal`：此處共實作偵測 POSIX 系統中斷訊號 (`SIGINT`)、終端尺寸變更訊號 (`SIGWINCH`) 的兩 watcher，分別進行以下動作：
  - `SIGINT`：觸發即代表使用者進行中斷訊號輸入，此時將所有 event loop 中斷，並通過 `terminated_cb` 函式切換為預設介面 (normal screen)。
  - `SIGWINCH`：用以偵測使用者是否改變現正使用終端機的長寬，以利及時進行動態 UI 處理[2]。

3.7. 回呼函式 (Callback Function) 功能說明

如小節 3.6 所述，本次專案總共實作 3 個回呼函式。具體功能敘述及實作如下。

3.7.1. `static void calc_cb (EV_P_ ev_io *w, int revents)`

在此 callback function 中，會進行以下操作：

1. 檢測是否為空字串或 `EOF`。
2. 利用一個 `char` 陣列 `buf` 作為中繼儲存，將輸入字串進行預存。
3. 判斷字串是否為 `"exit"` 或 `"quit"`，若是則將事件迴圈關閉並進入清理作業。
4. 將 `buf` 丟入 `calculate(char*)` 進行計算與結果印出作業。
5. 結束計算後重新印出命令提示符號 `"Expression\t>"`。

3.7.2. `static void terminated_cb(EV_P_ ev_signal *w, int revents)`

此函式將直接使事件迴圈關閉。

3.7.3. `static void resize_cb(EV_P_ ev_signal *w, int revents)`

此函式先偵測是否能正常讀取畫面寬度。若無問題則使時間顯示板更新，並使用最新的畫面寬度作為傾印 `'='` 字元之數量基礎。

3.7.4. `static void timer_cb(EV_P_ ev_timer *w, int revents)`

此函式將驅使時間顯示板做更新。

[2] 此專案中將自動偵測使用者目前使用 terminal 的寬度，動態調整時間顯示板列印 `'='` 字元的數量，並根據使用者是否調整寬度動態調整。由於此為 UI 優化部份，故不在主文贅述。

4. 複雜度分析

以下複雜度分析僅針對計算功能，針對長 n 的算式字串輸入進行分析，且不考慮結尾字元 `'\0'`。注意此處僅討論最差複雜度之狀況。

4.1. 空間複雜度分析

我們只討論初步認為可能因 n 影響大小的資料，所對應的空間複雜度。詳列如下：

- `num_stack operand`：由於 `num_stack` 定義中，我們固定 `content` 陣列尺寸為 128，其長度不隨 n 變化，即空間複雜度為 $O(1)$。
- `char_stack operator`：同上點， `content` 陣列尺寸不隨 n 變化，空間複雜度為 $O(1)$。
- `char num_buf[STRING_BUFFER_SIZE]`：由於以固定其長度為 `256`，故尺寸仍不隨 n 變化，空間複雜度為 $O(1)$。

綜上分析，結果加總後可得總空間複雜度為 $S(n) = O(1) + O(1) + O(1) = O(1)$■。

4.2. 時間複雜度

以下分項討論可能被 n 影響之算法時間複雜度：

- 字串處理與分類：程式處理了整個輸入字串的狀態，將其分類放入 `operator` 與 `operand` 兩個 stack。此一操作之最差時間複雜度對 n 而言為 $O(n)$。
- stack 操作：由於每一次的 push、pop、top 等操作均只操作一次，與輸入數量 n 完全無關，其時間複雜度無論型別，均為 $O(1)$。另外因每一次循環中進行 stack 操作的次數，最多與 n 成正比，故時間複雜度為 $O(n)$
- Operator 優先級判斷及運算：令 operator 數量為 $m \leq n$。該段程式雖在不同情況進入，但就整體程式流程而言，處理完後的運算子即遭到丟棄，故實際執行 `flush_operator()` 流程之次數為 m。故時間複雜度為 $O(m) = O(n)$。

綜上所述，結果加總後可得總時間複雜度為 $T(n) = O(n) + O(n) + O(n) = O(n)$■。

5. 測試結果

5.1. 基本測試資料與結果

使用測資如下表所陳：

#	Input	Output
1	3+5*2	13
2	100/5/2	10
3	8*4-6	26
4	123+45*6	393
5	10 + 20 * 3	70

表 1　測資一覽表。

測試結果如下圖所示。

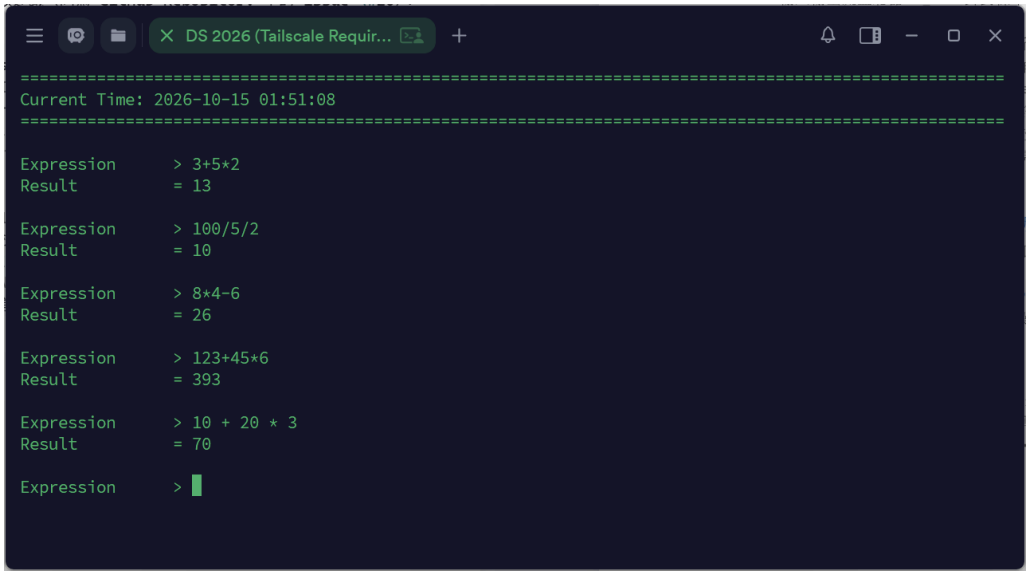


圖 2　測試結果，使用 Termius 終端機以 SSH 方式訪問遠端 Debian 主機執行 `calculator` 編譯結果。

5.2. 待修復問題

通過 OpenAI Codex 進行的 boundary test，目前已知現行版本有以下問題與修改建議（同為 GitHub Repository 中的 issue #20）：

- `*2 → 0`：運算元不足仍執行運算；依規格應回報 Invalid expression.。這是最嚴重的解析錯誤。
- `5 → Invalid expression.`：單一數值被拒絕。規格沒有明確要求支援單一數值，但建議確認是否希望接受。
- `2+`、`2/` → Invalid expression.：正確拒絕尾端缺運算元。
- `2**3`、`1..2`、`.5+1` → Invalid expression.：正確拒絕這些格式。
- `2^-2 → 0.25`：結果正確。
- `0^(-1)`、`10^1000 → inf`：沒有 NaN，但程式會直接輸出無限大。建議也檢查 isinf()，並回報適當錯誤。

- `64 位數字 → Invalid expression.`：不過解析器在數字長度超過 `MAX_NUM_STR_LENGTH` 時仍會繼續寫入 `num_buf`，可能越界。應在寫入前限制長度，超限就拒絕。